ChatGPT

Vector Databases and Retrieval Systems for Large-Scale AI Applications

Abstract

Vector databases and retrieval systems have become a foundational substrate for modern AI, supporting semantic search, recommendation, multimodal matching, long-term memory, and retrieval-augmented generation. Their emergence reflects the convergence of three research threads that were once studied separately: high-dimensional approximate nearest neighbor search, database systems for persistent and hybrid query processing, and neural information retrieval for learned representations. This survey synthesizes these threads into a systems-oriented account of how large-scale AI applications store, index, retrieve, update, and evaluate vectorized data. We review the algorithmic lineage from exact metric indexing and locality-sensitive hashing to product quantization, graph-based search, and memory–disk hybrids, then connect these methods to integrated database architectures that combine vector similarity with filters, joins, and transactional persistence. We further examine retrieval pipelines built on dense, sparse, hybrid, and multi-vector representations, with special attention to retrieval-augmented language models and multimodal systems. Across the survey, we emphasize the engineering tensions that now define the field: recall versus latency, compression versus fidelity, throughput versus freshness, vector similarity versus relational composability, and hardware efficiency versus portability. We argue that the center of gravity of the literature has shifted from stand-alone ANN libraries toward full vector database management systems and retrieval stacks that must support filtering, updates, distributed execution, GPU acceleration, and downstream reasoning workloads. This shift is especially visible in recent work on integrated relational/vector engines, distributed and RDMA-based search, GPU-native filtered retrieval, and benchmark suites designed for modern embeddings rather than classic vision descriptors. [1]

Introduction

The contemporary AI stack increasingly treats data as embeddings. Text passages, images, products, users, code fragments, molecules, and graph entities are mapped into dense or late-interaction vector spaces in which semantic similarity can be approximated by geometric proximity. This representational regime has altered the design requirements of data systems. Traditional DBMSs excel at exact predicates, aggregation, and transactional durability; neural retrieval systems excel at semantic matching over learned representations; approximate nearest neighbor systems excel at low-latency similarity search in high dimensions. Vector databases sit at the boundary of these traditions, seeking to expose semantic search as a database primitive while preserving the scale, reliability, and composability demanded by production AI systems [1–18, 102–117].

Historically, the core algorithmic problem was nearest neighbor search. Exact indexing structures such as k-d trees, metric trees, and cover trees provided strong geometric intuition but deteriorated in the high-dimensional regimes induced by learned embeddings [19–35]. Approximate nearest neighbor search therefore became the dominant formulation, with major method families including hashing, quantization, graph search, and hybrids that combine partitioning, compression, and multi-stage reranking [26–60]. What changed over the past several years is that the field no longer views ANN as an isolated algorithmic subproblem. Real deployments require persistent storage, incremental updates, hybrid scalar–vector filtering, distribution across nodes or accelerators, and integration into ranking or

generation pipelines. Recent VDBMS work explicitly addresses these concerns by embedding vector search into relational, graph, and distributed query engines rather than treating it as an external library call. [2]

A second reason the topic now matters more than before is the rise of retrieval-augmented AI. Dense passage retrievers, late-interaction document retrievers, and retrieval-augmented language models have changed the way external memory is used in NLP and multimodal reasoning [133–171]. In this setting, vector databases are not merely similarity indexes. They become the operational memory substrate of AI applications, responsible for grounding, freshness, provenance, personalization, and long-context efficiency. The storage and retrieval choices made at the systems layer can directly affect answer faithfulness, latency budgets, and operating cost.

This survey has three goals. First, it organizes the field by method families, from classical indexing to recent vector database architectures. Second, it bridges algorithmic and systems perspectives, showing how design choices in indexing, storage, hardware mapping, and query planning interact. Third, it identifies the open problems that remain difficult even after the recent wave of progress: filtered search under arbitrary predicates, streaming updates with stable recall, multi-vector and multimodal retrieval at scale, security of embedding spaces, and evaluation protocols that reflect current AI workloads rather than legacy descriptor datasets. A concise taxonomy is given in Table 1.

Method family	Core idea	Typical strength	Persistent limitation	Representative references
Exact and metric indexing	Prune search with geometric bounds in exact structures	Strong guarantees at low dimension	Breakdown in very high dimension	[19–35]
Hashing	Probabilistic bucketing of nearby vectors	Sublinear candidate generation	Recall instability and tuning sensitivity	[26–33]
Quantization	Compress vectors and compare approximate distances	Excellent memory efficiency	Quantization error and training overhead	[40–60]
Graph search	Traverse proximity graphs greedily	Outstanding recall–latency tradeoff in memory	Irregular access and difficult updates	[61–75]
Hybrid memory–disk search	Keep routing in RAM, bulk data on SSD	Billion-scale search under RAM limits	I/O sensitivity and maintenance complexity	[71–75, 109, 118–132]
Filtered and hybrid queries	Combine vector search with scalar predicates or joins	Aligns with production query needs	Selectivity-aware planning remains hard	[76–91, 102–117]
Retrieval stacks	Couple retrieval with ranking or generation	High end-task quality	Large storage footprint and evaluation complexity	[92–99, 133–171]

Methods

Algorithmic foundations of vector search

The classical point of departure is exact nearest neighbor indexing. k-d trees, ball trees, R-trees, M-trees, vantage-point trees, and cover trees all exploit spatial or metric structure to prune the search space [19–25]. In moderate dimensions or under favorable intrinsic dimensionality, these structures can be highly effective, and exact search remains relevant for small collections, low-latency reranking pools, and verification stages. Yet the shift from hand-crafted features to hundreds- or thousands-dimensional learned embeddings led to the familiar “curse of dimensionality,” motivating approximate search as the practical default [26, 27, 35].

Hashing methods were the earliest scalable approximation strategy. Theoretical work on approximate nearest neighbors and locality-sensitive hashing established sublinear retrieval guarantees under suitable similarity assumptions [26–29]. Multi-probe and data-dependent variants reduced memory blowup and improved empirical efficiency [30–33]. These approaches remain conceptually important, especially because modern dense retrieval training itself sometimes uses ANN-based hard-negative mining, as in ANCE [134]. However, pure LSH has gradually ceded ground in large-scale AI systems to quantization and graph approaches, which usually deliver better recall–latency tradeoffs on current embedding distributions [178, 179].

Quantization-based methods form the first major contemporary backbone of vector databases. Product quantization partitions the space into subvectors and replaces each with a learned codeword, making approximate distance computation cheap in memory and on SIMD/GPU hardware [40]. Subsequent work improved both coding quality and operating efficiency through reranking with source coding, optimized product quantization, Cartesian k-means, additive and composite quantization, locally optimized PQ, online PQ, compression-aware code layouts, and SIMD-accelerated distance computation [41–53]. Recent work such as ADSampling, RaBitQ, and practical theoretical quantizers revisits the approximation problem from the perspective of high-dimensional distance comparison itself, emphasizing error bounds and computational reliability rather than only codebook geometry [54–57]. Compression is no longer just a storage trick; it is also a query-time systems primitive.

Graph-based search is the other dominant family. Navigable small-world graphs, HNSW, EFANNA, FANNG, NSG, SSG, and Vamana-style monotonic graphs all exploit greedy routing on proximity neighborhoods [61–70]. Their empirical dominance in in-memory settings stems from a robust pattern: graph traversal can preserve very high recall at low latency when neighborhoods are well diversified and the entry strategy is strong [64, 70, 178, 179]. The downside is equally well known. Graphs induce irregular memory access, make compression less straightforward, and complicate insertion, deletion, and predicate-constrained traversal. Much of the recent literature can be interpreted as attempts to preserve graph quality while fixing one of these systems-level weaknesses.

Hybrid methods blur the traditional boundaries. DiskANN moved graph search partly to SSD, demonstrating that billion-point search could be performed on a single node with modest DRAM by combining graph routing, beam search, and disk-aware layout [71]. SPANN took a different route, using a memory–disk hybrid inverted methodology with balanced clusters and pruned posting-list access [73]. SPFresh, FreshDiskANN, and recent in-place update methods extend these constructions toward streaming corpora [72, 74, 75]. The modern field therefore no longer separates “algorithm” from “system” cleanly. Search quality depends on coding choices, which depend on hardware, which depend on update and persistence constraints. 3

Integrated vector database architectures

The vector database literature increasingly studies full systems rather than isolated retrieval kernels. Faiss provides the most influential research library abstraction, unifying flat search, IVF, PQ, HNSW-like constructions, and GPU implementations under a common framework [53, 101]. Yet the recent systems literature asks harder questions: how should vector search coexist with SQL operators, how should indexes be persisted and updated, which operators belong in the query optimizer, and how should structured predicates interact with ANN traversal?

Early integrated systems such as AnalyticDB-V and PASE explored hybrid processing of structured and unstructured data inside established database frameworks [102, 103]. Milvus made vector data management itself the central abstraction, foregrounding collection organization, index selection, and scalability [104]. VBASE advanced the integration agenda by proposing relaxed monotonicity as a unifying principle for online vector similarity search and relational queries, thereby making similarity predicates more compatible with iterator-based query execution [105]. SingleStore-V demonstrated that a distributed relational engine can expose vector search, filters, joins, and full-text features within one integrated data path [106]. Survey work from Pan, Wang, and Li crystallized this line of thought into the broader VDBMS research agenda [107, 108]. [4]

More recent systems focus on persistence, real-time behavior, and deployment structure. GaussDB-Vector targets large-scale persistent real-time vector search for LLM applications [109]. HARMONY studies scalable distributed vector databases for high-throughput ANN [110]. TigerVector shows that graph databases can also become hosts for vector search and hybrid graph–vector queries, which is particularly relevant for structured grounding and graph-enhanced RAG [111]. These systems collectively show that “vector database” now means more than a similarity index. It denotes a queryable, persistent, multi-operator system with support for data definition, lifecycle management, indexing, planning, and downstream application semantics.

The storage layer itself has become a research topic. PDX rethinks data layout for vector similarity search by vertically organizing dimensions across vectors, restoring the value of dimension-pruning methods that had lost ground against aggressively optimized horizontal scans [60]. Similarity Search in the Blink of an Eye with Compressed Indices demonstrates that vector compression and search-engine design can be co-designed to improve both throughput and memory footprint [58]. Tribase explores triangle-inequality-based pruning compression within a query engine rather than a stand-alone ANN benchmark setting [57]. These works indicate that physical design choices for vector data are now as important as high-level index families.

A crucial architectural theme is filtered and hybrid query processing. In production, vector similarity is rarely the sole predicate. Applications often impose timestamps, access-control labels, tenants, languages, regions, categories, or knowledge-graph constraints. Consequently, modern VDBMSs must decide when to pre-filter, post-filter, or fuse predicates into the ANN search process itself [76–91]. Recent panel and survey work explicitly identifies filtered vector search as a central systems challenge rather than an edge case. [5]

Filtered, range-constrained, and attribute-aware retrieval

Filtered ANN search extends the basic top-k vector problem by requiring candidates to satisfy structured conditions. A naïve pipeline retrieves nearest neighbors first and then discards candidates violating filters; this often fails for selective predicates because the candidate pool becomes too small or too biased. Conversely, pure pre-filtering can destroy ANN efficiency when the predicate does not admit a compact subset or when many labels overlap. Recent research therefore proposes a richer design space

involving label-aware partitions, predicate-aware graph navigation, segmented graphs, range-dedicated indices, and unified filter–vector abstractions [76–91].

Filtered-DiskANN is an important foundational system paper in this space, adapting graph algorithms to approximate nearest neighbor search with filters [86]. ACORN emphasizes predicate-agnostic search over embeddings and structured data [76]. Navigating Labels and Vectors proposes a unified approach to filtered ANN, while SeRF and IRangeGraph respectively design graph and range-dedicated structures tailored to scalar constraints [77–79]. UNIFY and later dynamic range-filtering work move toward unifying or dynamically maintaining these capabilities under shifting predicates and updates [80–82]. Meanwhile, attribute-constrained frameworks such as HQANN and related robust attribute-aware graph methods show that the problem is not confined to simple boolean filters; it includes multiattribute, range, and hybrid selection semantics [83–85].

The systems-level significance of this research is substantial. Filter-aware traversal changes candidate generation, distance computation order, memory locality, and even replication strategy in distributed engines. GPU-native filtered systems such as VecFlow suggest that massively parallel architectures can fundamentally alter the filter/search tradeoff when label-centric indexing is used [90]. Survey and benchmarking efforts from 2025–2026 have also made clear that classical benchmark datasets understate the difficulty of filtered search on modern transformer embeddings, especially in regimes where the query and corpus distributions differ or where selectivity varies widely across predicates. 6

Dynamic, distributed, and hardware-accelerated execution

Large-scale AI applications rarely operate on static corpora situated in the memory of a single CPU server. Streaming document collections, recommender inventories, user memories, enterprise knowledge bases, and multimodal repositories all require fresh updates, horizontal scaling, or specialized hardware support. This need has triggered a strong wave of research on dynamic updates, distributed execution, and accelerator-conscious design [71–75, 109–132].

Dynamic maintenance is difficult because many high-performing indices are structurally fragile. FreshDiskANN addressed streaming similarity search by storing the bulk of the graph on SSD and recent changes in RAM [72]. SPFresh proposed incremental in-place updates via lightweight local rebalancing, avoiding expensive global rebuilds [74]. Subsequent work on in-place graph-index updates continues to reduce the cost of deletion repair and neighbor maintenance [75]. These studies collectively show that freshness is not just a background maintenance concern; it is a first-class determinant of architectural choice.

For scaling out, recent systems move beyond shard-and-merge. HARMONY studies scalable distributed vector databases at high throughput [110]. CoTra uses RDMA to make distributed vector search more communication-efficient [116]. RED-ANNS uses RDMA-enabled distributed graph search to preserve a logically full graph across nodes and avoid the degradation caused by overly aggressive partitioning [117]. Work on CXL-based memory disaggregation likewise suggests that “single node versus distributed cluster” is no longer the only dichotomy; memory hierarchy and interconnect design increasingly shape feasible ANN operating points [118]. This line of research indicates that future vector databases may look more like disaggregated data systems than traditional search servers. 7

Accelerators are equally central. Faiss demonstrated the continuing relevance of GPU search for flat, compressed, and graph construction tasks [53]. Tagore studies GPU-accelerated graph indexing [119]. VecFlow and VecFlow-Chamfer extend GPU-native design to filtered and multi-vector retrieval [90, 91]. “To GPU or Not to GPU” takes a broader DBMS view, showing that the best CPU/GPU placement may differ for relational and vector components of hybrid SQL+VS workloads [115]. AiSAQ, ES4D, and related

in-storage or near-data approaches push the field further by relocating similarity computation toward storage devices themselves [120, 131]. The cumulative message is that hardware specialization is no longer peripheral. It materially changes which index families are attractive, how vectors should be laid out, and how query plans should be constructed. [8]

Retrieval models and application pipelines

A survey confined to storage/indexing would miss the reason vector databases matter in AI. Their operational significance comes from the retrieval pipelines they serve. Dense retrieval popularized the use of learned embeddings for first-stage document retrieval via dual encoders, with DPR and ANCE becoming seminal references for supervised dense retrieval [133, 134]. Subsequent unsupervised or weakly supervised work such as Contriever and corpus-aware pretraining reduced annotation dependence [137, 138]. Learned sparse retrieval, especially SPLADE and SPLADE v2, reminded the community that lexical sparsity remains competitive and often complementary to dense semantics [139, 140]. Thus, many practical systems now need to support hybrid sparse+dense retrieval rather than dense search alone.

Late-interaction and multi-vector retrieval create an even more demanding systems profile. ColBERT represents queries and documents as sets of token vectors and scores them with efficient late interaction rather than a single pooled embedding [96]. ColBERTv2 made this line of work more storage-conscious through residual compression and denoised supervision [97]. PLAID accelerated late-interaction retrieval through index-aware pruning [98]. More recent systems papers such as MUVERA, WARP, ESPN, and VecFlow-Chamfer reflect a distinct trend: the field is trying to make multi-vector retrieval approach the systems efficiency of single-vector ANN without giving up the effectiveness benefits of token-level matching [91–99]. This is a major development for vector databases because multi-vector methods can multiply storage footprints by an order of magnitude, fundamentally changing index and caching requirements.

Multimodal and vertical applications further broaden the picture. CLIP-style aligned vision–language embeddings enabled cross-modal search [9], while application papers such as VisRel and Amazon Shop the Look document industrial visual retrieval at scale [16, 17]. Graph-enhanced or knowledge-graph-integrated retrieval adds another layer by combining symbolic topology with vector similarity [111]. The conceptual message is that vector retrieval is no longer a single task. It is a family of application pipelines with distinct representation granularity, latency budgets, update rates, and composition needs.

Retrieval-augmented generation and learned external memory

Retrieval-augmented language modeling has turned vector retrieval into a core component of generative AI. REALM introduced retrieval-augmented language model pretraining with differentiable access to external corpora [151]. RAG formalized the now-standard retrieve-then-generate family for knowledge-intensive NLP [152]. FiD showed that generator architecture can benefit strongly from conditioning on many retrieved passages [153]. RETRO demonstrated retrieval from trillions of tokens for scaling language models with explicit memory [154]. Atlas advanced few-shot learning with retrieval-augmented language models [155]. REPLUG, Self-RAG, and related systems study black-box retrieval augmentation, adaptive retrieval, and critique-guided generation [156–159]. These works collectively establish that non-parametric memory is not merely a debugging aid; it is a central scaling strategy for factuality, provenance, and parameter efficiency. [9]

From a systems perspective, RAG introduces requirements that classical ANN benchmarks do not fully capture. First, corpora change quickly, so freshness and update cost matter. Second, retrieval is often followed by reranking, chunk stitching, or citation attribution, making top-k latency only one part of

the budget. Third, failures propagate differently: retrieval omission, retrieval mismatch, and generation unfaithfulness are distinct error modes. Fourth, query distributions can be highly out-of-distribution with respect to the indexed corpus, especially in domain-specific enterprise deployment. This is one reason new benchmark suites and analyses increasingly use transformer-generated embeddings and RAG-like workloads rather than only SIFT or GloVe vectors [168–174]. [10]

The literature also recognizes post-retrieval correction and attribution as part of the retrieval system, not merely part of the generator. RARR retrofits attribution and revision using retrieved evidence [161]. Work on parametric versus non-parametric memory, retrieval privacy, and evaluation frameworks such as ARES, CRAG, and RAGBench makes clear that end-to-end system quality cannot be inferred from ANN recall alone [165–170]. The retrieval substrate must therefore be studied jointly with generation, ranking, and evaluation.

Evaluation methodology and benchmark design

Evaluation in vector databases spans at least four layers: index quality, systems performance, retrieval effectiveness, and end-task utility. At the index layer, recall@k, latency, throughput, memory footprint, build time, and update cost remain standard [172, 173]. ANN-Benchmarks provided an influential comparative framework for in-memory ANN algorithms [172]. The Big ANN competitions extended this toward billion-scale and specialized tracks [173]. VIBE argues that benchmark datasets themselves must evolve because classical descriptor datasets are no longer representative of current embedding workloads or out-of-distribution query settings [174]. [11]

At the retrieval-effectiveness layer, IR benchmarks such as BEIR, MTEB, and MIRACL are crucial, particularly when vector databases serve dense, hybrid, or multilingual retrieval [141–143]. These benchmarks expose model-side variation that directly affects systems design: some embeddings produce cluster-friendly geometry, others produce harder candidate separation; some late-interaction retrievers justify enormous storage footprints through robustness. For RAG, benchmark suites such as ARES, CRAG, RAGBench, and T^2-RAGBench emphasize that evaluation must disentangle context relevance, answer faithfulness, answer relevance, and modality coverage [167–170]. The field is therefore moving toward a layered evaluation view in which ANN quality is necessary but insufficient.

Challenges and Prospects

The first challenge is freshness under high-quality indexing. The strongest ANN methods often assume global structural coherence. Yet production AI corpora evolve continuously: documents are added, revoked, re-embedded, deduplicated, or relabeled. Differential updates, stable deletes, and compaction without recall collapse remain difficult, especially for graph and clustered-disk indices [72, 74, 75]. This suggests an unresolved tension between static optimality and dynamic robustness. Future systems will likely need richer background-maintenance models, update-aware physical layouts, and adaptive cost models that expose freshness as a queryable constraint rather than a hidden maintenance detail.

The second challenge is composability. AI applications increasingly ask for queries that combine vector similarity with predicates, joins, text search, graph traversals, and tenant-specific constraints. The filtered-search literature has made real progress [76–91], but the general query-optimization problem remains open. Existing ANN-aware planners rarely reason jointly about predicate selectivity, vector index traversal cost, transfer cost across CPU/GPU boundaries, and downstream reranking stage budgets. As vector operators become first-class citizens in relational and graph engines, the next major step may be cost-based optimizers that treat vector search neither as a black box nor as a fixed physical operator.

The third challenge is representation granularity. Single-vector retrieval is systems-friendly, but multi-vector and multimodal retrieval often yield better task effectiveness [91–99]. Supporting token-level or region-level retrieval directly inside vector databases raises new issues: storage blowup, skewed posting distributions, heavy-tailed access patterns, and more complex score aggregation. Recent work on multi-vector engines is promising, yet the field lacks a mature theory of how storage layout, compression, and pruning interact with late-interaction score fidelity. Multi-vector-aware VDBMS design is therefore an important open frontier.

The fourth challenge is trustworthy and secure vector infrastructure. Retrieval-based LMs promise interpretability and updateability, but they also introduce privacy leakage, membership risks, and new attack surfaces [166, 182]. Embedding spaces can be poisoned, black-box attacked, or manipulated by adversarial examples; filters can leak sensitive label statistics; cross-tenant vector sharing raises access-control difficulties. These issues will likely drive a new subfield at the intersection of vector databases, privacy-preserving search, and secure AI systems. Federated vector similarity search and privacy analyses are early signals of this direction [114, 166].

The fifth challenge is evaluation realism and standardization. Benchmarks are improving, but no single suite captures the full range of workloads relevant to vector databases: static versus streaming corpora, dense versus late-interaction retrieval, low-selectivity versus high-selectivity filtering, in-distribution versus out-of-distribution queries, and retrieval-alone versus RAG end-tasks. Modern embeddings also change rapidly, so benchmark obsolescence is faster than in earlier ANN research [174]. The field would benefit from benchmark protocols that report not only recall–latency curves, but also update fragility, filter sensitivity, energy efficiency, failure recovery, and end-task impact.

The final challenge is unification. The literature still contains too many boundaries: ANN versus DBMS, retrieval model versus storage engine, GPU-optimized kernel versus declarative database operator, and benchmark leaderboards versus application utility. The most promising recent papers already cross these boundaries. A mature theory and practice of vector databases will likely emerge only when retrieval representations, physical design, query planning, and downstream reasoning are studied as parts of one system. [12]

Conclusion

Vector databases and retrieval systems now occupy a central role in large-scale AI. Their research lineage begins with nearest neighbor search, but their present-day importance lies in how they connect learned representations to persistent, composable, high-throughput data systems. The field has progressed from exact geometric structures and early hashing methods to quantization, graph search, and memory–disk hybrids; from stand-alone ANN libraries to integrated vector database management systems; and from document retrieval to multi-vector, multimodal, and retrieval-augmented generation pipelines. The most consequential shift is that vector search is no longer evaluated only as a geometric primitive. It is assessed as an operational component of AI systems that must jointly satisfy semantic quality, systems efficiency, freshness, provenance, and hybrid query semantics.

For researchers, the key lesson is that no single method family dominates under all objectives. Quantization remains indispensable for memory efficiency; graph methods remain exceptionally strong for recall at low latency; hybrid and disk-based systems matter for billion-scale deployment; integrated VDBMS designs matter for query composition; and retrieval-aware generation workloads impose constraints not visible in legacy ANN benchmarks. For practitioners, the strongest current designs are those that align representation choice, index family, storage layout, hardware placement, and downstream task requirements rather than optimizing any single layer in isolation. The likely future of

the field is therefore not "better ANN alone," but a deeper co-design of embeddings, indexes, query processors, accelerators, and evaluation protocols for AI-native data systems.

References

[1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.
[2] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and Their Compositionality," NeurIPS, 2013.
[3] J. Pennington, R. Socher, and C. Manning, "GloVe: Global Vectors for Word Representation," EMNLP, 2014.
[4] Q. Le and T. Mikolov, "Distributed Representations of Sentences and Documents," ICML, 2014.
[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv:1810.04805, 2018.
[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP, 2019, arXiv:1908.10084.
[7] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple Contrastive Learning of Sentence Embeddings," EMNLP, 2021, arXiv:2104.08821.
[8] G. Izacard, G. Caron, P. Hosseini, et al., "Unsupervised Dense Information Retrieval with Contrastive Learning," TMLR, 2022, arXiv:2112.09118.
[9] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML, 2021, arXiv:2103.00020.
[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.
[11] O. Barkan and N. Koenigstein, "Item2Vec: Neural Item Embedding for Collaborative Filtering," MLSP, 2016.
[12] M. Grohe, "word2vec, node2vec, graph2vec, x2vec: Towards a Theory of Vector Embeddings of Structured Data," PODS, 2020.
[13] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," CVPR, 2017.
[14] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," NeurIPS, 2017.
[15] C. Sutton, "The Gradient Flow of Neural Retrieval," arXiv preprint, 2023.
[16] F. Borisyuk, S. Malreddy, J. Mei, et al., "VisRel: Media Search at Scale," KDD, 2021.
[17] M. Du, A. Ramisa, A. Kumar KC, et al., "Amazon Shop the Look: A Visual Search System for Fashion and Home," KDD, 2022.
[18] J. J. Pan, J. Wang, and G. Li, "Vector Database Management Techniques and Systems," SIGMOD Companion, 2024.

[19] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," Communications of the ACM, 1975.
[20] S. M. Omohundro, "Five Balltree Construction Algorithms," International Computer Science Institute, 1989.
[21] A. Guttman, "R-Trees: A Dynamic Index Structure for Spatial Searching," SIGMOD, 1984.
[22] P. Ciaccia, M. Patella, and P. Zezula, "M-Tree: An Efficient Access Method for Similarity Search in Metric Spaces," VLDB, 1997.
[23] P. N. Yianilos, "Data Structures and Algorithms for Nearest Neighbor Search in General Metric Spaces," SODA, 1993.
[24] A. Beygelzimer, S. Kakade, and J. Langford, "Cover Trees for Nearest Neighbor," ICML, 2006.
[25] S. Dasgupta and Y. Freund, "Random Projection Trees and Low Dimensional Manifolds," STOC, 2008.
[26] P. Indyk and R. Motwani, "Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality," STOC, 1998.
[27] M. Datar, N. Immorlica, P. Indyk, and V. Mirrokni, "Locality-Sensitive Hashing Scheme Based on p-

Stable Distributions," SoCG, 2004.
[28] A. Andoni and P. Indyk, "Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor in High Dimensions," FOCS, 2006.
[29] Q. Lv, W. Josephson, Z. Wang, et al., "Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search," VLDB, 2007.
[30] Y. Weiss, A. Torralba, and R. Fergus, "Spectral Hashing," NeurIPS, 2008.
[31] M. Norouzi and D. J. Fleet, "Minimal Loss Hashing for Compact Binary Codes," ICML, 2011.
[32] Y. Gong and S. Lazebnik, "Iterative Quantization: A Procrustean Approach to Learning Binary Codes," CVPR, 2011.
[33] H. Liu, R. Wang, S. Shan, and X. Chen, "Deep Supervised Hashing for Fast Image Retrieval," CVPR, 2016.
[34] P. Ram and K. Sinha, "Revisiting k-d Tree for Nearest Neighbor Search," KDD, 2019.
[35] P. Schäfer, J. Brand, U. Leser, B. Peng, and T. Palpanas, "Fast and Exact Similarity Search in Less than a Blink of an Eye," arXiv:2411.17483, 2024.
[36] J. S. Beis and D. G. Lowe, "Shape Indexing Using Approximate Nearest-Neighbour Search in High-Dimensional Spaces," CVPR, 1997.
[37] S. Arya, D. Mount, N. Netanyahu, et al., "An Optimal Algorithm for Approximate Nearest Neighbor Searching in Fixed Dimensions," JACM, 1998.
[38] G. Navarro, "Searching in Metric Spaces by Spatial Approximation," The VLDB Journal, 2002.
[39] E. Chávez, G. Navarro, R. Baeza-Yates, and J. Marroquín, "Searching in Metric Spaces," ACM Computing Surveys, 2001.

[40] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," TPAMI, 2011.
[41] R. Tavenard, H. Jégou, L. Amsaleg, and H. Bouchard, "Searching in One Billion Vectors: Re-rank with Source Coding," ICASSP, 2011.
[42] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized Product Quantization," TPAMI, 2014.
[43] M. Norouzi and D. J. Fleet, "Cartesian k-Means," CVPR, 2013.
[44] A. Babenko and V. Lempitsky, "Additive Quantization for Extreme Vector Compression," CVPR, 2014.
[45] T. Zhang, C. Du, and J. Wang, "Composite Quantization for Approximate Nearest Neighbor Search," ICML, 2014.
[46] Y. Kalantidis and Y. Avrithis, "Locally Optimized Product Quantization for Approximate Nearest Neighbor Search," CVPR, 2014.
[47] X. Martinez, F. Perronnin, and H. Jégou, "LSQ: Lower Running Time and Higher Recall in Multi-codebook Quantization," ECCV, 2014.
[48] D. Xu, I. W. Tsang, Y. Zhang, and J. Yang, "Online Product Quantization," TKDE, 2018.
[49] L. Li and Q. Hu, "Optimized High Order Product Quantization for Approximate Nearest Neighbors Search," Frontiers of Computer Science, 2020.
[50] R. Wang and D. Deng, "DeltaPQ: Lossless Product Quantization Code Compression for High Dimensional Similarity Search," PVLDB, 2020.
[51] P. Sun, R. Guo, P. M. Kumar, et al., "Accelerating Large-Scale Inference with Anisotropic Vector Quantization," ICML, 2020, arXiv:1908.10396.
[52] F. André, A.-M. Kermarrec, and N. Le Scouarnec, "Quicker ADC: Unlocking the Hidden Potential of Product Quantization with SIMD," TPAMI, 2021, arXiv:1812.09162.
[53] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, 2019, arXiv:1702.08734.
[54] J. Gao and C. Long, "High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations," Proc. ACM Manag. Data, 2023, arXiv:2303.09855.
[55] J. Gao and C. Long, "RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search," Proc. ACM Manag. Data, 2024, arXiv:2405.12497.

[56] J. Gao, Y. Gou, Y. Xu, et al., “Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search,” arXiv:2409.09913, 2024.
[57] Q. Xu, J. Yang, F. Zhang, et al., “Tribase: A Vector Data Query Engine for Reliable and Lossless Pruning Compression Using Triangle Inequalities,” Proc. ACM Manag. Data, 2025.
[58] C. Aguerrebere, I. Bhati, M. Hildebrand, M. Tepper, and T. Willke, “Similarity Search in the Blink of an Eye with Compressed Indices,” PVLDB, 2023, arXiv:2304.04759.
[59] Y. Song, M. Qiao, F. Li, and D. Deng, “TRIM: Accelerating High-Dimensional Vector Similarity Search with Enhanced Triangle-Inequality-Based Pruning,” Proc. ACM Manag. Data, 2025, arXiv:2508.17828.
[60] L. Kuffo, E. Krippner, and P. Boncz, “PDX: A Data Layout for Vector Similarity Search,” Proc. ACM Manag. Data, 2025, arXiv:2503.04422.

[61] P. A. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, “Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs,” Information Systems, 2014.
[62] P. A. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, “Approximate Nearest Neighbor Search Small World Approach,” ICTA, 2011.
[63] P. A. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, “Scalable Distributed Algorithm for Approximate Nearest Neighbor Search Problem in High Dimensional General Metric Spaces,” SISAP, 2012.
[64] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” TPAMI, 2020, arXiv:1603.09320.
[65] B. Harwood and T. Drummond, “FANNG: Fast Approximate Nearest Neighbour Graphs,” CVPR, 2016.
[66] C. Fu and D. Cai, “EFANNA: An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on kNN Graph,” arXiv:1609.07228, 2016.
[67] M. Iwasaki, “Pruned Bi-directed k-Nearest Neighbor Graph for Proximity Search,” SISAP, 2016.
[68] M. Iwasaki and D. Miyazaki, “Optimization of Indexing Based on k-Nearest Neighbor Graph for Proximity Search in High-Dimensional Data,” arXiv:1810.07355, 2018.
[69] C. Fu, C. Xiang, C. Wang, and D. Cai, “Fast Approximate Nearest Neighbor Search With the Navigating Spreading-out Graph,” PVLDB, 2019.
[70] C. Fu, C. Wang, and D. Cai, “High Dimensional Similarity Search with Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility,” TPAMI, 2021.
[71] S. J. Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, “DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node,” NeurIPS, 2019.
[72] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, “FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search,” arXiv:2105.09613, 2021.
[73] Q. Chen, B. Zhao, H. Wang, et al., “SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search,” NeurIPS, 2021, arXiv:2111.08566.
[74] Y. Xu, H. Liang, J. Li, et al., “SPFresh: Incremental In-Place Update for Billion-Scale Vector Search,” SOSP, 2023, arXiv:2410.14452.
[75] H. Xu, H. Liang, J. Li, et al., “In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search,” arXiv:2502.13826, 2025.
[76] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, “ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data,” Proc. ACM Manag. Data, 2024.
[77] Y. Cai, J. Shi, Y. Chen, and W. Zheng, “Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search,” Proc. ACM Manag. Data, 2024.
[78] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, “SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search,” Proc. ACM Manag. Data, 2024.
[79] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, “IRangeGraph: Improvising Range-Dedicated Graphs for Range-Filtering Nearest Neighbor Search,” Proc. ACM Manag. Data, 2024.
[80] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, “UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search,” arXiv:2412.02448, 2024.

[81] F. Zhang, M. Jiang, G. Hou, et al., "Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search," Proc. ACM Manag. Data, 2025.
[82] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic Range-Filtering Approximate Nearest Neighbor Search," PVLDB, 2025.
[83] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint," NeurIPS, 2023.
[84] W. Wu, J. He, Y. Qiao, et al., "HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints," CIKM, 2022.
[85] X. Xu, C. Li, Y. Wang, and Y. Xia, "Multiattribute Approximate Nearest Neighbor Search Based on Navigable Small World Graph," Concurrency and Computation: Practice and Experience, 2020.
[86] S. Gollapudi, et al., "Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters," WWW, 2023.
[87] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis," arXiv:2602.11443, 2026.
[88] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of Filtered Approximate Nearest Neighbor Search over the Vector-Scalar Hybrid Data," arXiv:2505.06501, 2025.
[89] P. Iff, P. Bruegger, M. Chrapek, M. Besta, and T. Hoefler, "Benchmarking Filtered Approximate Nearest Neighbor Search Algorithms on Transformer-Based Embedding Vectors," arXiv:2507.21989, 2025.
[90] J. Xi, C. Mo, B. Karsin, A. Chirkin, M. Li, and M. Zhang, "VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs," Proc. ACM Manag. Data, 2025, arXiv:2506.00812.
[91] C. Mo, B. Karsin, P. Adams, and M. Zhang, "VecFlow-Chamfer: A GPU-Based Data Management System for High-Performance Multi-Vector Search on Superchips," Proc. ACM Manag. Data, 2026.
[92] R. Jayaram, J. Lee, and V. Mirrokni, "MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings," OpenReview, 2025, arXiv:2405.19504.
[93] J. L. Scheerer, M. Zaharia, C. Potts, G. Alonso, and O. Khattab, "WARP: An Efficient Engine for Multi-Vector Retrieval," arXiv:2501.17788, 2025.
[94] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," SIGIR, 2020, arXiv:2004.12832.
[95] K. Santhanam, O. Khattab, J. Saad-Falcon, et al., "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," NAACL, 2022, arXiv:2112.01488.
[96] J. Santhanam, C. Potts, and O. Khattab, "PLAID: An Efficient Engine for Late Interaction Retrieval," CIKM, 2022, arXiv:2205.09707.
[97] E. Lassance and S. Clinchant, "An Efficiency Study for Late Interaction Retrieval Models," SIGIR, 2023.
[98] S. Rajpurkar and A. Chidambaram, "Efficient Constant-Space Multi-Vector Retrieval," arXiv preprint, 2025.
[99] S. Mackenzie, G. Zuccon, and J. Mackenzie, "ESPN: Memory-Efficient Multi-Vector Information Retrieval," SIGIR, 2024.

[100] J. Johnson, M. Douze, H. Jégou, and M. Aumüller, "The Faiss Library," arXiv:2401.08281, 2024.
[101] C. Wei, B. Wu, S. Wang, et al., "AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data," PVLDB, 2020.
[102] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension," SIGMOD, 2020.
[103] J. Wang, et al., "Milvus: A Purpose-Built Vector Data Management System," SIGMOD, 2021.
[104] Q. Zhang, S. Xu, Q. Chen, et al., "VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity," OSDI, 2023.
[105] C. Chen, C. Jin, Y. Zhang, et al., "SingleStore-V: An Integrated Vector Database System in SingleStore," PVLDB, 2024.
[106] J. J. Pan, J. Wang, and G. Li, "Survey of Vector Database Management Systems," The VLDB Journal, 2024.

[107] J. Sun, G. Li, J. Pan, J. Wang, Y. Xie, R. Liu, and W. Nie, “GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications,” PVLDB, 2025.
[108] Q. Xu, F. Zhang, C. Li, et al., “HARMONY: A Scalable Distributed Vector Database for High-Throughput Approximate Nearest Neighbor Search,” Proc. ACM Manag. Data, 2025, arXiv:2506.14707.
[109] S. Liu, Z. Zeng, L. Chen, et al., “TigerVector: Supporting Vector Search in Graph Databases for Advanced RAGs,” SIGMOD Companion, 2025, arXiv:2501.11216.
[110] Z. Zhu, Z. Fan, Y. Zeng, et al., “FedSQ: A Secure System for Federated Vector Similarity Queries,” PVLDB, 2024.
[111] V. Mageirakos, J. André, M. Kabić, B. Wu, Y. Chronis, and G. Alonso, “To GPU or Not to GPU: Vector Search in Relational Engines,” arXiv:2605.15957, 2026.
[112] X. Zhi, M. Chen, X. Yan, et al., “CoTra: Towards Efficient and Scalable Distributed Vector Search with RDMA,” Proc. ACM Manag. Data, 2026.
[113] Y. Chen, K. Zhang, S. Chen, et al., “RED-ANNS: An RDMA-Enabled Distributed Framework for Graph-Based Approximate Nearest Neighbor Search,” PVLDB, 2026.
[114] J. Jang, H. Choi, H. Bae, et al., “CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search,” USENIX ATC, 2023.
[115] Z. Li, X. Ke, Y. Zhu, et al., “Scalable Graph Indexing Using GPUs for Approximate Nearest Neighbor Search,” Proc. ACM Manag. Data, 2025, arXiv:2508.08744.
[116] A. Sindhwani, P. Kumar, and L. Massoulié, “SOAR: Improved Indexing for Approximate Nearest Neighbor Search,” NeurIPS, 2023, arXiv:2404.00774.
[117] X. Zhong, Y. Cheng, Q. Chen, et al., “VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search,” PVLDB, 2025.
[118] M. D. Manohar, et al., “ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms,” SPAA, 2024.
[119] K. Iwabuchi, T. Steil, B. Priest, and R. Pearce, “SOLANET: Distributed Neighbor Graph Construction on GPU-Accelerated Systems,” arXiv preprint, 2026.
[120] Z. Wang, M. Qiao, W. Zhou, F. Li, and D. Deng, “AiSAQ: All-in-Storage ANNS with Product Quantization for Billion-Scale Similarity Search,” arXiv:2404.06004, 2024.
[121] J. Kim, et al., “ES4D: Accelerating Exact Similarity Search for High-Dimensional Data on SSDs,” ICCD, 2022.
[122] C. Aguerrebere, et al., “Similarity Search in the Blink of an Eye with Compressed Indices,” PVLDB, 2023.
[123] H. Gu, et al., “ScaDANN: A Scalable Disk-Based Graph Indexing Method for Approximate Nearest Neighbor Search,” CEUR Workshop Proceedings, 2025.
[124] J. Zhang, et al., “BAMG: A Block-Aware Monotonic Graph Index for Disk-Based Approximate Nearest Neighbor Search,” arXiv:2509.03226, 2025.
[125] M. Cao, et al., “PageANN: Scalable Disk-Based Approximate Nearest Neighbor Search,” arXiv: 2509.25487, 2025.
[126] X. Xie, et al., “DSANN: Approximate Nearest Neighbor Search of Large-Scale Vector Data in Distributed Storage,” arXiv:2510.17326, 2025.
[127] J. Sun and C. Long, “SRS: Solving c-Approximate Nearest Neighbor Queries in High Dimensional Euclidean Space with a Tiny Index,” PVLDB, 2015.
[128] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng, “Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search,” PVLDB, 2015.
[129] D. Cai, “A Revisit of Hashing Algorithms for Approximate Nearest Neighbor Search,” arXiv: 1612.07545, 2019.
[130] Y. Sun, W. Wang, J. Qin, Y. Zhang, and X. Lin, “SRS: Solving c-Approximate Nearest Neighbor Queries in High Dimensional Euclidean Space with a Tiny Index,” PVLDB, 2014.
[131] J. Qin, W. Wang, C. Xiao, Y. Zhang, and Y. Wang, “High-Dimensional Similarity Query Processing for Data Science,” KDD, 2021.

[132] K. Zoumpatianos, K. Echihabi, and T. Palpanas, "New Trends in High-D Vector Similarity Search: AI-Driven, Progressive, and Distributed," PVLDB, 2021.

[133] V. Karpukhin, B. Oğuz, S. Min, et al., "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020, arXiv:2004.04906.
[134] L. Xiong, C. Xiong, Y. Li, et al., "Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval," ICLR, 2021, arXiv:2007.00808.
[135] Y. Gao and J. Callan, "Condenser: a Pre-training Architecture for Dense Retrieval," EMNLP, 2021, arXiv:2104.08253.
[136] Y. Gao and J. Callan, "coCondenser: a Dense Retriever with Unsupervised Corpus-Aware Contrastive Pre-training," arXiv:2112.09118, 2021.
[137] G. Izacard, et al., "Unsupervised Dense Information Retrieval with Contrastive Learning," TMLR, 2022, arXiv:2112.09118.
[138] K. Lee, et al., "SEED-Encoder: Self-Training Dense Retrieval with Contrastive Sampling," arXiv preprint, 2022.
[139] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," SIGIR, 2021, arXiv:2107.05720.
[140] T. Formal, C. Lassance, B. Piwowarski, and S. Clinchant, "SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval," arXiv:2109.10086, 2021.
[141] N. Thakur, N. Reimers, A. Rücklé, et al., "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," NeurIPS Datasets and Benchmarks, 2021, arXiv:2104.08663.
[142] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," EACL, 2023, arXiv:2210.07316.
[143] X. Zhang, N. Thakur, O. Ogundepo, et al., "Making a MIRACL: Multilingual Information Retrieval Across a Continuum of Languages," TACL, 2023, arXiv:2210.09984.
[144] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval Without Relevance Labels," ACL, 2023, arXiv:2212.10496.
[145] J. M. Su, et al., "HyDE: Precise Zero-Shot Dense Retrieval with Hypothetical Document Embeddings," ACL Findings, 2023.
[146] J. Mackenzie, S. Zhuang, and G. Zuccon, "Exploring the Representation Power of SPLADE Models," arXiv:2306.16680, 2023.
[147] J. Dai and J. Callan, "Context-Aware Term Expansion for Sparse Retrieval," SIGIR, 2020.
[148] O. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv:1901.04085, 2019.
[149] R. Nogueira, W. Yang, K. Cho, and J. Lin, "Multi-Stage Document Ranking with BERT," arXiv: 1910.14424, 2019.
[150] R. Nogueira, W. Yang, J. Lin, and K. Cho, "Document Expansion by Query Prediction," arXiv: 1904.08375, 2019.

[151] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," ICML, 2020, arXiv:2002.08909.
[152] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020, arXiv:2005.11401.
[153] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," EACL, 2021, arXiv:2007.01282.
[154] S. Borgeaud, A. Mensch, J. Hoffmann, et al., "Improving Language Models by Retrieving from Trillions of Tokens," ICML, 2022, arXiv:2112.04426.
[155] G. Izacard, P. Lewis, M. Lomeli, et al., "Atlas: Few-shot Learning with Retrieval Augmented Language Models," JMLR, 2023, arXiv:2208.03299.
[156] W. Shi, S. Min, M. Yasunaga, et al., "REPLUG: Retrieval-Augmented Black-Box Language Models," NAACL, 2024, arXiv:2301.12652.
[157] A. Asai, S. Min, Z. Zhong, and D. Chen, "Retrieval-Based Language Models and Applications," ACL

Tutorial Abstracts, 2023.
[158] A. Asai, Z. Zhong, R. Chen, et al., “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” ICLR, 2024, arXiv:2310.11511.
[159] M. Xie, et al., “RA-DIT: Retrieval-Augmented Dual Instruction Tuning,” arXiv preprint, 2023.
[160] W. Chen, H. Hu, X. Chen, et al., “MuRAG: Multimodal Retrieval-Augmented Generator for Open Question Answering over Images and Text,” EMNLP, 2022.
[161] L. Gao, Z. Dai, P. Pasupat, et al., “RARR: Researching and Revising What Language Models Say, Using Language Models,” ACL, 2023, arXiv:2210.08726.
[162] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Generalization through Memorization: Nearest Neighbor Language Models,” ICLR, 2020, arXiv:1911.00172.
[163] A. Mallen, A. Asai, V. Zhong, et al., “When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories,” ACL, 2023, arXiv:2212.10511.
[164] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems,” NAACL, 2024, arXiv:2311.09476.
[165] X. Yang, K. Sun, H. Xin, et al., “CRAG: Comprehensive RAG Benchmark,” NeurIPS Datasets and Benchmarks, 2024, arXiv:2406.04744.
[166] R. Friel, M. Belyi, and A. Sanyal, “RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems,” arXiv:2407.11005, 2024.
[167] J. Strich, et al., “T^2-RAGBench: Text-and-Table Benchmark for Evaluating Retrieval-Augmented Generation,” arXiv:2506.12071, 2025.
[168] S. Liu, X. Li, Z. Liu, et al., “Judge as A Judge: Improving the Evaluation of Retrieval-Augmented Generation through the Judge-Consistency of Large Language Models,” Findings of ACL, 2025, arXiv: 2502.18817.
[169] Y. Huang, et al., “Privacy Implications of Retrieval-Based Language Models,” arXiv:2305.14888, 2023.
[170] J. Liu, et al., “RETA-LLM: A Retrieval-Augmented Large Language Model Toolkit,” arXiv: 2306.05212, 2023.
[171] S. Wu, et al., “Retrieval-Augmented Generation for Natural Language Processing: A Survey,” Language Resources and Evaluation, 2024, arXiv:2407.13193.

[172] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms,” Information Systems, 2019, arXiv:1807.05614.
[173] H. V. Simhadri, M. Aumüller, M. Douze, et al., “Results of the Big ANN: NeurIPS’23 Competition,” arXiv:2409.17424, 2024.
[174] E. Jääsaari, V. Hyvönen, M. Ceccarello, T. Roos, and M. Aumüller, “VIBE: Vector Index Benchmark for Embeddings,” arXiv:2505.17810, 2025.
[175] Y. Han, C. Liu, and P. Wang, “A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge,” arXiv:2310.11703, 2023.
[176] L. Ma, et al., “A Comprehensive Survey on Vector Database,” arXiv:2310.11703, 2023.
[177] J. J. Pan, J. Wang, and G. Li, “Survey of Vector Database Management Systems,” The VLDB Journal, 2024.
[178] M. Wang, X. Xu, Q. Yue, and Y. Wang, “A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search,” PVLDB, 2021, arXiv:2101.12631.
[179] I. Azizi, K. Echihabi, and T. Palpanas, “Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art,” Proc. ACM Manag. Data, 2025, arXiv:2502.05575.
[180] Y. Chronis, V. Mageirakos, M. Kabić, et al., “Filtered Vector Search: State-of-the-Art and Research Opportunities,” PVLDB, 2025.
[181] K. Echihabi, K. Zoumpatianos, and T. Palpanas, “New Trends in High-D Vector Similarity Search: AI-Driven, Progressive, and Distributed,” PVLDB, 2021.
[182] H. Li, et al., “Can You Trust the Vectors in Your Vector Database? Black-Box Adversarial Attack on Vector Search,” arXiv:2604.05480, 2026.

[183] M. Li, et al., “Attribute Filtering in Approximate Nearest Neighbor Search,” arXiv:2508.16263, 2025.
[184] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, “Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis,” arXiv:2602.11443, 2026.

---

1 2 12 https://link.springer.com/article/10.1007/s00778-024-00864-x
https://link.springer.com/article/10.1007/s00778-024-00864-x

3 https://papers.nips.cc/paper/9527-rand-nsg-fast-accurate-billion-point-nearest-neighbor-search-on-a-single-node
https://papers.nips.cc/paper/9527-rand-nsg-fast-accurate-billion-point-nearest-neighbor-search-on-a-single-node

4 https://www.vldb.org/pvldb/vol13/p3152-wei.pdf
https://www.vldb.org/pvldb/vol13/p3152-wei.pdf

5 6 https://www.vldb.org/pvldb/vol18/p5488-caminal.pdf
https://www.vldb.org/pvldb/vol18/p5488-caminal.pdf

7 https://dl.acm.org/doi/10.1145/3749167
https://dl.acm.org/doi/10.1145/3749167

8 https://arxiv.org/abs/2605.15957
https://arxiv.org/abs/2605.15957

9 https://arxiv.org/abs/2002.08909
https://arxiv.org/abs/2002.08909

10 https://arxiv.org/abs/2505.17810
https://arxiv.org/abs/2505.17810

11 https://arxiv.org/abs/1807.05614
https://arxiv.org/abs/1807.05614